

CDS View 課程練習 10：Custom Entity

Lecture

這一課要教什麼

到目前為止所有的 CDS View，資料來源都是 `select from <某張表>`——底層一定有一張真實的資料庫表。但有些情境資料根本不在資料庫表裡：呼叫外部系統（RFC / Web Service）、即時計算的統計資料、甚至完全是程式邏輯生成的內容。**CDS Custom Entity**（`define custom entity`）就是為這種情境設計的：宣告一組欄位結構，但資料完全由一個 **ABAP 類別負責提供**，不綁定任何資料庫表。

這一課會建一個刻意簡單、但能完整證明「資料真的不是從表來的」的範例：一個「機隊狀態」清單，三筆固定資料完全在 ABAP 程式碼裡生成。

語法元素講解

① **define custom entity**：語法跟 `define view` 不同，不是 `select from`，是直接宣告欄位型別：

```
define custom entity <名稱>
{
  key <欄位1> : <型別>;
    <欄位2> : <型別>;
}
```

② **@ObjectModel.query.implementedBy**：這個 annotation 把 Custom Entity 綁定到一個 ABAP 類別，格式是 `'ABAP:<類別名稱>'`：

```
@ObjectModel.query.implementedBy: 'ABAP:ZCL_CDS10_STATUS_QUERY'
```

③ **IF_RAP_QUERY_PROVIDER** 介面：Query Provider 類別要實作這個介面，直接讀這系統的標準介面定義確認精確簽章（不是憑印象猜）：

```
INTERFACE if_rap_query_provider PUBLIC.
  METHODS select IMPORTING io_request TYPE REF TO if_rap_query_request
                                io_response TYPE REF TO if_rap_query_response
                                RAISING cx_rap_query_provider.
ENDINTERFACE.
```

`io_request`（`IF_RAP_QUERY_REQUEST`）用來查詢「這次呼叫要什麼」（`is_data_requested()`、`is_total_numb_of_rec_requested()`、`get_filter()`、`get_paging()`等）；`io_response`（`IF_RAP_QUERY_RESPONSE`）用 `set_data( it_data )` 回傳資料、`set_total_number_of_records( ... )` 回傳總筆數。

完整範例

Custom Entity：

```
@EndUserText.label: 'CDS10: Fleet Status (Custom Entity, non-DB source)'
@ObjectModel.query.implementedBy: 'ABAP:ZCL_CDS10_STATUS_QUERY'
define custom entity ZI_CDS10_FLEET_STATUS
{
  key StatusCode : abap.char(10);
    StatusText : abap.char(40);
    SortOrder : abap.int4;
}
```

Query Provider 類別 (重點節錄) :

```
METHOD if_rap_query_provider~select.
  DATA lt_data TYPE STANDARD TABLE OF ty_status WITH EMPTY KEY.

  " 沒有任何資料庫表參與，資料完全在 ABAP 裡生成
  lt_data = VALUE #(
    ( statuscode = 'ACTIVE'      statustext = 'Aircraft in active service'
      sortorder = 1 )
    ( statuscode = 'MAINTENANCE' statustext = 'Aircraft undergoing maintenance'
      sortorder = 2 )
    ( statuscode = 'RETIRED'     statustext = 'Aircraft retired from service'
      sortorder = 3 )
  ).

  IF io_request->is_data_requested( ).
    io_response->set_data( lt_data ).
  ENDIF.

  IF io_request->is_total_numb_of_rec_requested( ).
    io_response->set_total_number_of_records( lines( lt_data ) ).
  ENDIF.
ENDMETHOD.
```

⚠ 重大發現：Custom Entity 完全不能用 Open SQL 直接查詢

這一課的第一個實測動作，就是故意在驗證程式裡直接對 **ZI_CDS10_FLEET_STATUS** 下一句 **SELECT**，想看看會不會查到資料——結果在編譯階段就直接被擋下來：

```
Entities like "ZI_CDS10_FLEET_STATUS" cannot be used here.
```

這不是執行期才發現查不到資料，是 ABAP 編譯器**根本不允許**把 Custom Entity 當作 Open SQL 的資料來源。官方文件 **ABENCDS_F1_CUSTOM_QUERY** 講得很清楚：**@ObjectModel.query.implementedBy** 綁定的類別，只有「透過框架存取，例如 RAP Query Engine」時才會被呼叫——這代表 Custom Entity 唯一合法的存取管道是**透過 RAP 框架**（EML、Service Binding / OData），純 Open SQL 這條路完全不通。

這是這一課最重要的觀念：Custom Entity 不是「另一種可以直接查的 View」，它是專門設計給 RAP 應用場景（例如當作 Value Help 的資料來源、或當作某個 RAP BO 的唯讀查詢節點）用的，跟 cds01 ~ cds09 教的、可

以隨意用 Open SQL 查詢的一般 CDS View 是完全不同的使用方式。

⚠ 這系統的驗證限制，跟這一課發明的一個繞過技巧

正常情況下，要驗證 Query Provider 類別真的會被呼叫、真的會回傳資料，需要透過 Service Binding (Eclipse-only，這系統很多課程都遇過這個限制) 走真正的 OData 呼叫——但 `programrun` 無頭執行環境沒辦法發起真正的 OData 請求 (本課程系列已經記錄過「自我呼叫 HTTP 在 `programrun` 底下會卡住」的限制)。

這一課用了一個新的驗證技巧繞過這個限制：既然 `IF_RAP_QUERY_PROVIDER` 只是一個標準介面，自己寫兩個最小化的 **Mock** 類別 (分別實作 `IF_RAP_QUERY_REQUEST` / `IF_RAP_QUERY_RESPONSE`)，直接呼叫 `ZCL_CDS10_STATUS_QUERY` 的 `select` 方法，不透過任何 RAP 框架：

```
DATA(lo_request) = NEW lcl_mock_request( ). " 自己寫的 Mock，回答「資料有被要求」
DATA(lo_response) = NEW lcl_mock_response( ). " 自己寫的 Mock，把 set_data 傳進來的內容印出來
DATA(lo_provider) = NEW zcl_cds10_status_query( ).

lo_provider->if_rap_query_provider~select(
    io_request = lo_request
    io_response = lo_response ).
```

這個技巧驗證的範圍要誠實說清楚：這樣測試只能證明「Query Provider 類別本身的邏輯正確」(真的會生成 3 筆資料、真的會呼叫 `set_data` / `set_total_number_of_records`)，**不能證明**「這個 Custom Entity 透過真正的 RAP Query Engine / OData 呼叫時也會正確運作」——後者仍然需要建 Service Definition、Service Binding (Eclipse-only)，並在 Eclipse 用 Data Preview 或 Postman 實際打一次 OData 端點才能完整驗證。但這個 **Mock** 測試技巧依然有實際價值：它至少證明了「業務邏輯本身沒有寫錯」，把「RAP 框架有沒有正確接上」這個問題獨立出來，之後真的要接 Service Binding 時，如果查不到資料，就能確定問題出在框架接線，不是 Query Provider 類別本身的邏輯。

Eclipse ADT 建立 CDS View：Step by Step

1. 建類別 `ZCL_CDS10_STATUS_QUERY`：對著 `$TMP` 套件右鍵 → New → ABAP Class，實作 `IF_RAP_QUERY_PROVIDER`，內容照抄上面的完整範例
2. 建 Custom Entity `ZI_CDS10_FLEET_STATUS`：New → Other ABAP Repository Object → **Data Definition**，注意 **Templates** 畫面這次要選跟 **Custom Entity** 相關的模板 (不是一般的 **Define View**)，或直接手動打完整內容
3. 順序：類別要先啟用成功，Custom Entity 的 `@ObjectModel.query.implementedBy` annotation 才能正確解析
4. 這一課沒有 Data Preview 可以直接驗證 (Custom Entity 不支援一般 View 的 Data Preview 走 Open SQL 那條路)，要驗證得靠下一步：建 Service Definition + Service Binding，用 Eclipse 內建的 Preview / Swagger UI 實際打一次

這一課學到的東西，接下來會怎麼用

- `cds13` (Value Help)：Custom Entity 常見的實務用途之一，就是當作 Value Help 的資料來源 (尤其資料不在資料庫表裡的情境)

- 這一課發明的「Mock Request/Response 測試技巧」，任何未來要驗證 RAP Query Provider 邏輯、又不想依賴完整 OData 呼叫的情境都能重用

Eclipse ADT Step by Step (重點回顧)

1. 先建 `ZCL_CDS10_STATUS_QUERY` (實作 `IF_RAP_QUERY_PROVIDER`)
2. 再建 `ZI_CDS10_FLEET_STATUS` (`define custom entity` + `@ObjectModel.query.implementedBy`)
3. 兩者都啟用成功後，Custom Entity 本身無法用一般 Open SQL / Data Preview 驗證，要靠 Service Binding (Eclipse-only) 或本課教的 Mock 測試技巧

學習目標

- 能寫出 `define custom entity` 的欄位宣告語法 (跟 `define view` 的 `select from` 完全不同)
- 能寫出 `@ObjectModel.query.implementedBy: 'ABAP:<類別>'` 並實作 `IF_RAP_QUERY_PROVIDER` 介面 (`select` 方法，`io_request/io_response`)
- 能講出「Custom Entity 完全不能用 Open SQL 直接查詢」這個實測發現的具體錯誤訊息，並解釋根本原因 (只有透過 RAP 框架存取才會觸發 Query Provider 類別)
- 能講出「Mock Request/Response 直接呼叫」這個驗證技巧測試了什麼、沒測試到什麼，不誇大驗證範圍
- 知道 Custom Entity 的實務定位：專門給資料來源不是資料庫表的場景 (外部系統、動態計算)，且是 RAP 生態系的一環，不是一般查詢用的 CDS View

物件清單

物件	名稱	型別
Query Provider 類別	<code>ZCL_CDS10_STATUS_QUERY</code>	CLAS/OC
CDS Custom Entity	<code>ZI_CDS10_FLEET_STATUS</code>	DDLS/DF
驗證程式 (含 Mock 測試)	<code>ZR_CDS10_DEMO</code>	PROG/P

全部物件都在 `$TMP` 套件，已建立並啟用 (讀取 active 版本內容與寫入內容逐字相符)。

動手練習 (留待後續補做)

1. 修改 `ZCL_CDS10_STATUS_QUERY`，改成回傳 5 筆資料 (自己設計內容)
2. 在 Eclipse 對著 `ZI_CDS10_FLEET_STATUS` 右鍵，找找看有沒有專屬的 Custom Entity 相關精靈/選項，記錄畫面實際長怎樣 (這一課純靠 ADT API / 手動輸入完成，沒有實測過 Eclipse 精靈本身)
3. 試著建一個 Service Definition + Service Binding，實際走一次 OData Preview，親自驗證這一課只用 Mock 技巧驗證過的部分

驗證方式

`ZR_CDS10_DEMO` 透過 `programrun` 無頭驗證兩件事：① Open SQL 無法查詢 Custom Entity (編譯期錯誤，已記錄具體訊息)；② 用自製 Mock Request/Response 直接呼叫 Query Provider 類別，驗證邏輯正確：

```
=== Attempt 1: plain Open SQL SELECT against the custom entity (expected to fail) ===
Compile-time result: "Entities like ZI_CDS10_FLEET_STATUS cannot be used here."
```

```
=== Attempt 2: call ZCL_CDS10_STATUS_QUERY directly with mock request/response objects
===
ACTIVE      Aircraft in active service                1
MAINTENANC  Aircraft undergoing maintenance           2
RETIRED     Aircraft retired from service              3
Total number of records captured (set_total_number_of_records):      3
Row count captured (set_data, rows printed above):      3
=== Sanity check: 3 status rows generated purely in ABAP, no DB table involved ===
MATCH: query provider class correctly generated 3 rows without reading any database
table
```

動手練習的驗證方式：Eclipse 啟用成功即可；如果做了 Service Binding，用 Eclipse Preview 貼截圖給我核對。

思考題

1. 如果 `ZCL_CDS10_STATUS_QUERY` 的 `select` 方法內部真的去呼叫一個外部 RFC 或 HTTP API 取得資料，這一課學到的「Mock Request/Response」測試技巧還能不能用？能驗證到什麼程度？
2. `io_request->is_data_requested( )` 跟 `io_request->is_total_numb_of_rec_requested( )` 是兩個獨立的旗標——你能想像什麼情境下框架只需要總筆數、不需要實際資料嗎？（提示：想想看 Fiori Elements 列表畫面的分頁機制）
3. 這一課的資料完全寫死在 ABAP 程式碼裡，如果要讓它真正動態（例如依 `io_request->get_filter( )` 篩選），程式碼大致要怎麼調整？
4. Custom Entity 跟 cds01 ~ cds09 教的一般 CDS View，最本質的差異是什麼？如果你要幫同事解釋「什麼時候該用 Custom Entity」，你會怎麼講？

答案

見 `zcl_cds10_status_query.clas.abap`、`zi_cds10_fleet_status.ddls.abap`、`zr_cds10_demo.prog.abap`。SAP 端物件：`ZCL_CDS10_STATUS_QUERY`（Query Provider 類別）、`ZI_CDS10_FLEET_STATUS`（Custom Entity）、`ZR_CDS10_DEMO`（驗證程式）。動手練習由你在 Eclipse 動手建立，稍後補做，沒有固定答案快照。